研究日期: 2026-03-28 | 模型: Claude Opus 4.6 (Thinking)

# Doramagic v12.1.1 管线架构重构设计报告

> 彻底告别 `singleshot.py` 时代脆弱的黑盒线性管线，基于 8 条血泪教训与 Haystack 2.0/LangGraph/DSPy 启发，重新设计的确定性上下文路由架构。

---

## 一、 核心架构范式转移

当前管线是固定线性映射（Profile → Discovery → Extraction → Synthesis → Compile → Validate → Deliver），无条件分支。

**改造方向：基于数据流的确定性状态机 (Data-Driven State Machine)** 借鉴 Haystack 2.0（数据流图）的思想，结合我们原有的 12 状态 `FlowController`，废弃固定的 `TRANSITIONS` 字典，引入 `next_state(current_state, context) -> State` 确定性路由逻辑。控制流骨架完全由代码（而非 LLM）决定。

---

## 二、 8 大核心设计模块 (v12.1.1 必须实现)

### 1. 输入路由器 (Input Router)

**解决问题：** 痛点 1（URL 盲搜浪费）和痛点 2（模糊请求不追问）。 **设计方案：** 在 `PHASE_A (Profile)` 阶段，使用小模型（如 Haiku/MiniMax，强制输出 JSON）对输入进行 4 象限快速分类，产生确定性路由标志： * **URL 直达** ( `has_exact_urls=True` ): 包含合法代码仓库链接 → 直接路由到 `PHASE_CD_FANOUT` ，跳过 `PHASE_B` 搜索。 * **明确项目名** ( `is_named_entity=True` ): 如 "React, Vue" → 路由到 `PHASE_B` 执行精确匹配模式。 * **泛领域探索** ( `is_domain_exploration=True` ): 如 "前端框架" → 路由到 `PHASE_B` 执行广度搜索模式。 * **意图模糊** ( `is_vague=True` ): 意图不清 → 路由到 `CLARIFY` 状态，管线挂起向用户请求补充信息。

### 2. 状态机改造与条件边 (Conditional Edges)

**解决问题：** 痛点 3（不同类型项目走同一路径）及固定跳转的局限。 **设计方案：** 引入基于代码的条件判断。例如在 `PHASE_CD` 的 Stage 0 (repo_facts) 后加入判定： * **条件判定**：若 Markdown 占比 > 80% 且无核心代码逻辑 → 触发 `is_collection=True` 。 * **条件路由**：针对 awesome-list 类项目，路由至轻量级元数据提取路径（仅提取推荐理由），跳过深度的抽象语法树分析和源码挖掘。

### 3. 扇出/扇入与数据隔离 (Fan-out / Fan-in)

**解决问题：** 痛点 4（一个失败污染全局）。 **设计方案：** * **扇出 (Fan-out)**：从 `PHASE_B` 拿到 $N$ 个目标后，状态机进入 `PHASE_CD_FANOUT`。启动 $N$ 个独立的 `CD_Executor` 子进程，每个拥有独立的 `staging/repo_{uuid}/` 工作域（遵守教训 #7 域隔离与 #2 subprocess 隔离）。 * **扇入 (Fan-in)**：`PHASE_E (Synthesis)` 作为汇聚节点。只要有 $\ge 1$ 个子进程成功，就继续执行。失败分支被记录在 `context.failed_repos` 中，实现局部失败不影响全局。

### 4. 信息传递契约 (Information Passing Contract)

**解决问题：** 教训 #5（信息饥饿导致下游质量暴跌）。 **设计方案：** 严格定义上下游 Payload Schema，杜绝仅传递自然语言或 URL： * **A → B**: `SearchConstraints(must_include_tech=[], min_stars=int, domain_keywords=[])` * **B → CD**: `RepoContext(url, commit_sha, targeted_directories=[], specific_framework_bricks=[])` * **CD → E**: `List[ConceptCard, RuleCard]`，`DSD_Scores`，`EvidenceTags` * **E → F**: `SynthesisNarrative(conflict_matrix, core_philosophy, unsaid_wisdom)`，明确指导编译器排版。

### 5. 质量反馈环与模块化编译 (Quality Feedback Loop)

**解决问题：** 痛点 5（编译占耗时 51%）和痛点 6（编译完才发现质量差）。 **设计方案：** 汲取 DSPy 模块化优化思想： * **编译前置快检 (Pre-flight Check)**：`PHASE_E` 结束时用规则引擎秒级检查卡片数和冲突率，不达标直接降级，避免进入高耗时 `PHASE_F`。 * **模块化编译**：将大单体编译拆分为并行编译 `advisor-brief.md`、`CLAUDE.md`、`.cursorrules`。 * **针对性修复环**：`PHASE_G (Validate)` 若报错 `{"pass": false, "weakest_module": "rules"}`，且 `retry_count < 1`，则仅将该模块发回 `PHASE_F_PARTIAL` 重新编译，而非重跑全程。

### 6. 降级交付体系 (Graceful Delivery)

**解决问题：** 痛点 7（任何失败 = 整个 run 失败）。 **设计方案：** 全局异常捕获转化为多级交付物： * **Level 1 (最佳)**: `PHASE_H` 正常交付完整 SKILL 包。 * **Level 2 (编译失败)**: 若 `PHASE_F/G` 崩溃，交付 `PHASE_E` 产出的原始卡片库及 `cross_project_synthesis.json`，附免责声明。 * **Level 3 (提取部分失败)**: 若 `PHASE_E` 综合失败，交付存活的 `PHASE_CD` 独立报告。 * **Level 4 (发现失败)**: 若 `PHASE_CD` 全部失败，交付 `PHASE_B` 产出的目标项目清单供用户手动查阅。

### 7. 进度事件架构 (Progress Events)

**解决问题：** 痛点 8（4 分钟黑盒无反馈）。 **设计方案：** 引入独立于日志的事件总线，实时输出 `run_events.jsonl`，Schema 如下：

```
{
  "timestamp": 1711600000.123,
```

```
  "trace_id": "run-8f3a",
  "phase": "PHASE_CD",
  "sub_task": "repo_tandoor",
  "status": "PROGRESS", // START | PROGRESS | END | ERROR
  "progress_pct": 0.45,
  "message": "正在剥离 Tandoor 的数据库反模式 (Rule: DR-02)..."
}
```

OpenClaw adapter 将消费该文件，以 3–5 秒间隔向客户端（Telegram等）发送状态更新。

---

## 三、 总结与后续演进

通过上述 7 大改造，我们将 `Doramagic v12.1.1` 从一条脆弱的直线，升级为了一个具备**条件感知、局部容错、并行加速和实时可观测**的工业级数据流管线。控制流骨架由代码严密锁定（避免 LLM 逃逸），而智能体（LLM）专注于节点内的数据处理与信息提取。

**后续演进预留：** 未来可拓展**多级反馈环**（允许 `PHASE_CD` 发现线索时退回 `PHASE_B` 补充搜索），并将**降级交付物知识持久化**（向量化存入本地库实现 O(1) 召回）。当前 v12.1.1 优先落地核心的确定性路由架构。